Wind Farm Layout Optimization

Assignment — Theoretical Base

MetaLab Suite

Problem #1

October 10, 2026

Abstract

This document defines the Wind Farm Layout Optimization (WFLO) problem that you must solve. It gives the decision variables, the topographical and spacing constraints, the simplified *Jensen* wake model, the turbine power curve, and the objective function—the expected farm power. It is the *theory*; the technical instructions for generating instances and evaluating layouts are in the companion guide `wind_farm_layout_optimization/README.md`. The simulator, the instance generator and an interactive GUI are provided; the *metaheuristic is your task*. A single random-search example and an object-oriented skeleton are included only to show the interface and to give you a structure to implement.

Contents

1 Assignment brief **1**

2 The model **1**
2.1 Decision variables 1
2.2 Topographical and engineering constraints 2
2.3 The wind rose 2
2.4 Simplified Jensen wake model 2
2.5 Turbine power and objective 3
2.6 Why the problem is hard 3

3 What you are given **3**

4 Starting point: random search **4**

5 Deliverables **4**

1 Assignment brief

You are given a piece of land (a *terrain polygon*), a set of forbidden areas (*exclusion zones*), a number of turbines N, and a *wind rose* describing how often the wind blows from each direction and how fast. You must decide **where to place the N turbines** so as to maximise the farm's **expected electrical power**.

The obstacle is the *wake effect*: a turbine extracts energy from the flow and leaves a slower wake behind it, so a turbine standing downwind of another produces less. Spreading turbines out reduces interference but is limited by the available terrain and by a minimum-spacing rule.

You must implement **one trajectory-based metaheuristic** (simulated annealing, tabu search, iterated local search, ...) that calls the provided simulator. You are expected to design

the encoding, the neighbourhood and the acceptance rule yourselves. *The only algorithm supplied is the random-search example of Section 4; the object-oriented skeleton is there for you to fill in.*

2 The model

2.1 Decision variables

A layout is a set of N turbine positions in the plane,

$$X = \{\, x_1, x_2, \ldots, x_N \,\}, \qquad x_j = (x_j^{(1)}, x_j^{(2)}) \in \mathbb{R}^2, \tag{1}$$

to be chosen within the terrain. In code, a layout is simply a list of N pairs "`[[x, y], ...]`". The search space is continuous; the constraints below make its feasible set non-convex.

2.2 Topographical and engineering constraints

Let $\Omega \subset \mathbb{R}^2$ be the terrain polygon. The feasible set is the intersection of the following:

(C1) Inside the terrain. Every turbine lies in Ω (in code, a point-in-polygon test):

$$x_j \in \Omega, \qquad j = 1, \ldots, N. \tag{2}$$

(C2) Outside the exclusion zones. Forbidden polygons $\{Z_m\}_{m=1}^{M}$ (buildings, protected areas, ...) must be avoided:

$$x_j \notin \bigcup_{m=1}^{M} Z_m, \qquad j = 1, \ldots, N. \tag{3}$$

(C3) Minimum spacing. Turbines keep a centre-to-centre distance of at least γ rotor diameters D,

$$\|x_i - x_j\| \;\geq\; s_{\min} = \gamma\, D, \qquad 1 \leq i < j \leq N, \tag{4}$$

with γ given in the instance (typically 3–4). Constraint (C3) is what removes the "stack them all upwind" trivial solution and makes Ω effectively non-convex.

2.3 The wind rose

The climate is summarised by a finite set of S scenarios $\{(\theta_s, U_s, p_s)\}_{s=1}^{S}$, where θ_s is the bearing the wind blows *towards*, U_s its free-stream speed, and $p_s \in [0,1]$ its relative frequency ($\sum_s p_s = 1$). The wind travel unit vector is $n_s = (\cos\theta_s, \sin\theta_s)$.

2.4 Simplified Jensen wake model

For two turbines i and j define the along-wind and cross-wind separations

$$x_{ij} = (x_j - x_i) \cdot n_s, \qquad y_{ij} = \left|(x_j - x_i) \cdot n_s^{\perp}\right|. \tag{5}$$

The Jensen model [1] assumes a linear, top-hat wake whose radius grows with downstream distance,

$$R_w(x) = \frac{D}{2} + k\, x, \tag{6}$$

where k is the wake-decay constant. Turbine j suffers a fractional velocity deficit from turbine i when it lies in that wake ($x_{ij} > 0$ and $y_{ij} \leq R_w(x_{ij})$):

$$\delta_{ij} = (1 - \sqrt{1 - C_T}) \left(\frac{D}{D + 2k\, x_{ij}} \right)^2, \qquad \delta_{ij} = 0 \text{ otherwise}, \tag{7}$$

with C_T the thrust coefficient. The factor $1 - \sqrt{1 - C_T}$ is twice the axial induction factor. Superposing the contributions of all upwind turbines in root-sum-square form, the speed seen by turbine j under scenario s is

$$u_j^{(s)} = U_s \left(1 - \sqrt{\sum_{i \in \mathcal{I}_j} \delta_{ij}^2} \right)_+, \qquad \mathcal{I}_j = \{\, i : x_{ij} > 0,\ y_{ij} \le R_w(x_{ij}) \,\}, \tag{8}$$

where $(\cdot)_+ = \max(\cdot, 0)$.

2.5 Turbine power and objective

A turbine converts wind speed into power through the cubic power curve (with cut-in u_{in}, rated speed u_r, cut-out u_{out} and rated power P_r):

$$P(u) = \begin{cases} 0, & u < u_{\text{in}} \text{ or } u \ge u_{\text{out}}, \\ P_r \Big(\dfrac{u - u_{\text{in}}}{u_r - u_{\text{in}}} \Big)^3, & u_{\text{in}} \le u < u_r, \\ P_r, & u_r \le u < u_{\text{out}}. \end{cases} \tag{9}$$

The farm power under scenario s is the sum over turbines, $P_{\text{total}}^{(s)}(X) = \sum_{j=1}^{N} P(u_j^{(s)})$, and the **objective** (to be maximised) is the expected power over the rose:

$$\boxed{\bar{P}(X) = \sum_{s=1}^{S} p_s \, P_{\text{total}}^{(s)}(X) = \sum_{s=1}^{S} p_s \sum_{j=1}^{N} P\Big(u_j^{(s)}(X) \Big).} \tag{10}$$

The simulator reports $\bar{P}$ together with the *actual efficiency*, defined as the ratio of the expected power to the installed capacity,

$$\eta(X) = \frac{\bar{P}(X)}{N \, P_r} \in [0, 1], \tag{11}$$

and the annual energy production $\text{AEP}(X) = 8760 \, \bar{P}(X)$ (in the instance's power units).

2.6 Why the problem is hard

The objective (10) is non-separable (a turbine's output depends on all turbines upwind of it), non-smooth (the wake indicator $y_{ij} \le R_w(x_{ij})$ and the kinks of (9) are non-differentiable), and multimodal (each of the $\binom{N}{2}$ pairwise wake interactions contributes local optima). Together with the non-convex feasible set of Section 2.2, this defeats gradient-based solvers and calls for *trajectory-based metaheuristics*.

3 What you are given

The following components are provided (the first two must not be modified):

- the **simulator** (`simulator_wflo.py`) — the `WindFarmEnvironment` class implements Section 2, exposing `evaluate_power_output(layout, wind_scenario)` and the aggregate `evaluate(layout)`;
- the **instance generator** (`generate_instances_wflo.py`) — builds the `toy` and `hard` instances (terrain, exclusion zones, N, wind rose);
- an interactive **Streamlit GUI** (`app_wflo.py`) — run a solver and draw the terrain, the turbines and the wake cones;

- the **only algorithm you are given**, a random search (`random_search_wflo.py`), together with an **object-oriented skeleton** (`student_wflo.py`) for you to implement;
- the **autograder backend** (`ingestion.py`, `score.py`).

All commands to generate instances and evaluate layouts are in the technical guide: `wind_farm_layout_optimization/README.md`.

A layout is "feasible" when it satisfies (C1)–(C3); the simulator reports this for you, together with the objective and the efficiency.

4 Starting point: random search

The only algorithm supplied is a plain random search — the weakest possible method. Use it to learn the interface; the real work is yours.

Listing 1: The provided random-search example (object-oriented).

```python
class RandomSearchSolver(BaseWFLOSolver):
    name = "random_search"

    def solve(self, time_limit=None):
        best_layout = self.env.random_layout(seed=self.seed)
        best_value = self.score(best_layout)
        self.record(best_value)
        for _ in range(self.iterations):
            layout = self.env.random_layout(seed=...)
            value = self.score(layout)
            if value > best_value:
                best_layout, best_value = layout, value
            self.record(best_value)
        return best_layout
```

Following the same pattern, you implement `MySolver.solve` in `student_wflo.py`: produce a candidate `layout`, score it with `self.score(layout)`, keep the best one, and call `self.record(...)` so the GUI can plot the convergence curve. The design of the move (neighbourhood) and of the acceptance rule is entirely up to you.

Your skeleton (object-oriented)

`BaseWFLOSolver` is the abstract interface every solver must respect, and `MySolver` is an empty subclass with `TODO` markers. Implementing it means deciding, for instance, the initial solution (random or greedy), the perturbation that produces a neighbouring layout, and the acceptance rule (hill climbing, simulated annealing with a cooling schedule, tabu search, ...).

5 Deliverables

1. A Python module implementing **one trajectory-based metaheuristic** that uses `WindFarmEnvironment` without modifying it.
2. Your best layouts for the `toy` and `hard` instances, saved under `solutions/`.
3. A short report containing: the convergence curve, the best expected power and efficiency, and an explanation of how your method handles the terrain, exclusion and spacing constraints.

References

[1] N. O. Jensen, "A note on wind generator interaction," Risø National Laboratory, Roskilde, Denmark, Tech. Rep. Risø-M-2411, 1983.

[2] I. Katić, J. Højstrup, and N. O. Jensen, "A simple model for cluster efficiency," in *Proc. European Wind Energy Association Conference and Exhibition*, Rome, Italy, 1986, pp. 407–410.

[3] G. Mosetti, C. Poloni, and B. Diviacco, "Optimization of wind turbine positioning in large wind farms by means of a genetic algorithm," *Journal of Wind Engineering and Industrial Aerodynamics*, vol. 51, no. 1, pp. 105–116, 1994.

[4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.